编译系统设计赛技术报告

摘要

本项目设计并实现了一套面向 SysY 语言和 AArch64/Cortex-A53 目标平台的编译系统。系统采用分层结构来组织前端语义、SSA 形式的中端优化以及结构化 Machine IR 后端，在保证源语言语义和 AAPCS64 调用约定的基础上，研究循环、矩阵计算、递归过程、跨函数状态以及目标机器代码之间的协同优化方法。

中端以控制流、支配、循环、数据流、别名和副作用分析为依据，对二维仿射更新、幂等迭代、重复仿射复制、有序原地递推和递归状态转移建立可验证的语义摘要，并据此实施维度折叠、冗余迭代消除、递推闭式化、完整参数键记忆化、状态本地化和数据布局变换。同时针对不能任意交换计算次序的矩阵程序，系统进一步实现严格保持依赖顺序的 wavefront 调度、列打包和多种面板微内核，以提高数据复用率并降低访存开销。

后端围绕 AArch64 指令选择、寻址模式、寄存器分配、NEON 向量化、栈访问配对和预取进行优化。所有晚期变换均在带类型操作数、基本块引用和控制流关系的 Machine IR 上完成，汇编打印阶段仅负责确定性展开，不使用汇编文本、标签名称或测试程序标识作为优化输入。

实验结果表明，所用方法能够显著降低若干计算密集型程序的执行时间：代表性矩阵测试获得约 11.8×、7.1× 和 5.2× 的加速，三角动态规划测试获得 3.23× 加速，递归状态程序获得 1.81× 加速。同时在正确性方面，当前版本通过全部 140 项 AArch64 正确性测试，并对 60 项性能测试的最终汇编和运行行为进行了完整检查。说明我们所实现的基于结构化语义证明的变换，可以在保持通用性和合规性的同时，有效降低程序的计算量、访存量和控制流开销。

关键词：编译优化；AArch64；循环变换；递归记忆化；矩阵计算；Machine IR

目录

摘要 **I**

第一章 问题定义与技术路线 **1**
1.1 设计目标 1
1.2 分层协同优化方法 1

第二章 系统架构与基础设施 **2**
2.1 总体架构 2
2.2 前端与 IR 3
2.3 中端分析基础 3
2.4 AArch64 后端流水线 3

第三章 算法级循环与数组优化 **4**
3.1 二维仿射更新的行摘要降维 4
3.2 幂等迭代与最后可观察迭代 4
3.3 重复仿射复制的按需反向解析 5
3.4 循环规范化与局部变换 5

第四章 有序矩阵计算与 AArch64 微内核 **6**
4.1 为什么不能直接套用 GEMM 6
4.2 多级有序面板 6
4.3 Wavefront、列打包与预取 7
4.4 条件矩阵归约与 parity unswitch 7

第五章 递归、状态摘要与记忆化 **8**
5.1 递归优化分层 8
5.2 尾递归增量摘要 8
5.3 闭式 helper 与状态本地化 8

第六章 Machine IR 与 AArch64 代码生成 **9**
6.1 图着色寄存器分配 9
6.2 地址与循环机器优化 9
6.3 结构化 post-RA 向量化 9

第七章 完整优化体系 **10**
7.1 过程间、函数与状态优化 10
7.2 循环与算术优化 10
7.3 数组、内存与布局优化 10
7.4 AArch64 目标优化 10

第八章 实验设计与性能结果 **11**
8.1 评测方法 11
8.2 代表性官方结果 12
8.3 代表性优化案例分析 12
8.3.1 01_mm：从指令优化到算法降维 12
8.3.2 h-5：围绕真实缓存层级逐步演进 12
8.3.3 many_mat：有序依赖下的面板化 13
8.4 负结果如何反向约束设计 13
8.5 当前版本验证结论 13

第九章 总结与后续工作 **14**

附录 A 默认 -O1 Pass 完整索引 **15**

附录 B 后端阶段与关键开关 **17**
B.1 后端阶段 17
B.2 代表性消融开关 17

参考资料与实现披露 **18**

第 1 章

问题定义与技术路线

1.1 设计目标

系统面向 SysY2026 程序和 AArch64 比赛目标，优化等级为 `-O1`。设计目标按优先级排序如下：

1. **语义正确和比赛合规**。优化只能依赖 IR/Machine IR 语义、CFG、支配、循环、数据流、别名、逃逸、效果分析和目标 ISA；不能依赖样例名、路径、用户符号名、生成标签拼写、公开输入输出或运行时探测。

2. **削减动态工作量**。优先寻找可把 $O(n^3)$ 降为 $O(n^2)$、把重复迭代折叠为一次、把指数级递归变为有界状态计算的机会。

3. **可解释、可消融、可回退**。每项优化都应具备结构化前置条件、独立环境开关、匿名正反例和全量门禁。

1.2 分层协同优化方法

传统教学编译器通常以 Mem2Reg、常量折叠、CSE、DCE、LICM 和简单指令选择为主。本系统保留这些基础能力，但性能突破主要来自以下三层协同：

表 1.1: 三层协同优化框架

层次	主要问题	代表实现
语义层	是否能少算、跳过或总结整段计算	行摘要降维、幂等迭代折叠、dead-prefix、仿射复制按需解析、递推闭式/分块总结
数据层	如何消除重复状态和访存	全局标量本地化、循环标量提升、Store-to-Load 转发、影子表、列打包、producer-consumer 融合
机器层	如何映射到 Cortex-A53 的寄存器、NEON 与缓存	Machine LSR、MADD/MSUB、LDP/STP、NEON 面板、parity unswitch、PRFM、图着色分配

第 2 章

系统架构与基础设施

2.1 总体架构



图 2.1: 编译器总体架构

2.2 前端与 IR

首先由 Lexer 生成带源码位置的 Token；然后递归下降 Parser 构建声明、表达式、控制流、数组和函数调用 AST；语义阶段维护嵌套符号表，检查重复定义、未定义引用、返回类型、数组维度和调用参数。

IR 采用显式基本块、指令、操作数、Phi 和终结指令，整数运算遵循 i32/i64 位宽语义。Module 保存函数、全局对象、初始化器和目标信息。优化框架提供函数/模块 pass、CFG 重写、值替换、Phi 修复、支配与自然循环分析。

2.3 中端分析基础

- **CFG 与支配：**识别唯一预头、回边、出口、必经路径、SCC 和循环嵌套；
- **SSA/use-def：**追踪归纳变量、递推状态、地址 GEP、返回值和纯表达式切片；
- **对象身份：**把 GlobalObject、alloca、FrameIndex 和派生 GEP 视为不透明对象，名称不参与优化判定；
- **过程间效果：**汇总 readnone、只读、读写对象、未知调用和递归调用图；
- **范围与回绕：**区分有符号比较、无符号地址、i32 模 2^{32} 回绕、除余向零截断和移位宽度；
- **fallback：**无法完成证明时保留原 CFG，静态不足时插入范围、别名、行程或上下文 guard。

2.4 AArch64 后端流水线

后端入口位于 `TargetBackend.cpp` 的 `AArch64Backend::generateAssembly`。该函数不是若干独立优化的简单串接，而是围绕表示层级安排的四段式流水线：首先把 SSA 风格 IR 降低为带类型、对象身份和显式 CFG 的 Machine IR；随后在虚拟寄存器阶段完成地址、循环和控制流规范化；寄存器分配后再进行依赖物理寄存器、栈帧和最终布局的结构化变换；最后由汇编打印器确定性地展开 Machine IR 与 AArch64 pseudo。图 **??** 给出了与当前源码一致的调用顺序。

2.4.1 Machine IR 边界与目标降低

`Lowering::lowerModule` 通过共享的 target lowering 将 Module 中的全局对象、函数签名、基本块、Phi、调用和访存降低到 Machine IR。此时并不急于生成汇编文本，而是保留 `ValueType`、`ObjectType`、`FrameIndex`、`StackObject`、虚拟寄存器以及标签操作数。AArch64 的 `TargetInstrAdapter` 统一给出 call、return、条件分支的判定，以

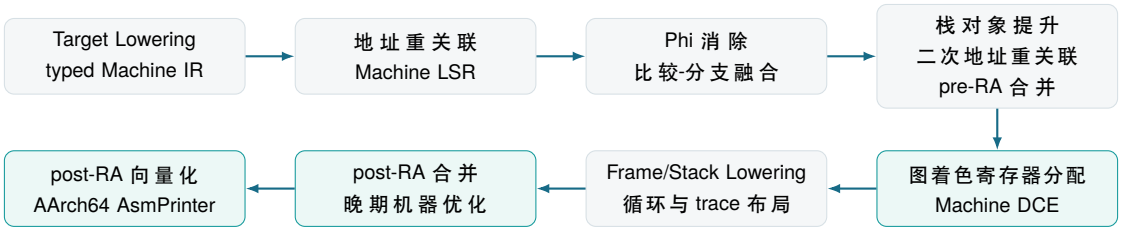


图 2.2: AArch64 后端实际调用顺序。上排主要操作虚拟寄存器，下排从寄存器分配开始处理物理寄存器、栈帧和最终块布局。

及每条机器指令的 use/def 枚举和替换接口。这样，后续活跃性、CFG、冲突图和循环分析不依赖汇编字符串，也不会把块名或用户符号的拼写作为优化条件。

这层表示同时承担两类指令。普通算术、访存和分支继续使用目标无关 opcode；已经完成严格结构证明的目标特化操作则使用 `aarch64.*` pseudo，并显式携带寄存器、对象、步长、范围和控制流出口。前者便于复用通用机器优化，后者使复杂 NEON 或记忆化序列可以在 AsmPrinter 中一次性、可审计地展开。

2.4.2 虚拟寄存器阶段：规范化优先于分配

寄存器分配前的目标是减少地址计算、规范化 CFG，并尽可能暴露可合并模式。

地址重关联被有意执行两次。第一次为 Machine LSR 提供规范的基址、下标和 scale；Phi 消除、栈对象提升又会生成新的 copy 和地址机会，因此在指令合并前再次运行。Machine LSR 只在可证明的自然循环上把乘法式地址改为递增指针，并处理 $iv + c$、相邻访问共享 pointer Phi 以及非二次幂 stride。Phi 消除则把 SSA 汇合转换为边上的并行复制；复制环通过临时虚拟寄存器打破，无法安全放入前驱时拆分边，从而在进入寄存器分配前彻底去除 Phi。

2.4.3 寄存器分配、栈帧与块布局

图着色分配器对整数类和浮点类分别求解。冲突边来自块级活跃性和逐指令反向扫描；copy、Phi 边以及调用参数/返回值形成带循环深度权重的 affinity，使合并和 ABI 寄存器偏好进入同一个代价模型。跨调用活跃值优先选择 callee-saved，着色失败时分配 spill slot，并在 use/def 点插入 reload/store。调用点还依据精确活跃集合保存必要的 caller-saved，而不是无条件保存全部易失寄存器。AArch64 保留 `x15--x18`、`x29--x30` 和 `s16--s18` 等专用寄存器，避免与汇编降低及晚期 pseudo 的 scratch 需求冲突。

分配后，Machine DCE 在物理寄存器层重新计算 live-in/live-out，删除已被 coalescing、重写或 spill 遗留的死亡定义。FrameLowering 首先扫描调用，确定超过 `x0--x7` 或 `s0--s7` 的传出参数区；StackLowering 再布局返回地址、可选帧指针、callee-saved、局部对象、spill 与传出参数区，并将总帧大小向 16 字节对齐。叶函数且无栈对象时完全省略栈帧；直接递归函数在不需要动态调栈且正文不引用帧指针时也可省去帧指针的保存、建立与恢复。最后的 loop placement 和 trace placement 只重排基本块，不改变

CFG 语义，并让循环回边或高收益后继尽量利用顺序落空，减少显式跳转。

2.4.4 post-RA 结构化优化与 NEON 降低

物理寄存器确定后，`AArch64InstructionCombining::runPostRA` 完成四类局部收尾：把相邻 i32 读取合并为 `LDP`；把对齐、连续且位于对象边界内的栈清零串压缩为 `aarch64.zero.stack.range`；把倒计数循环的减一、比较零和分支改为 `SUBS+B.NE`；重定向仅含同寄存器 copy 的空操作块。所有规则都显式检查目标寄存器冲突、偏移编码范围、对象大小和标志位透明性。

随后，`AArch64LateMachineOptimization` 在最终 CFG 和物理寄存器上识别“正整数反复折半”状态机。它只接受结构完整的除二、奇偶分支、深度累加与返回模式，并进一步验证参数类型、寄存器占用、全局上下文用途以及不存在调用和外部写入；满足条件时才生成折半闭式 pseudo 和模块内单一的尾递归记忆化站点。该位置安排在布局后，可以直接利用真实寄存器与 fall-through 关系，同时把复杂展开留给打印器。

`AArch64PostRAVectorization` 同样不是通用的猜测式循环向量化器，而是三组严格的结构迁移：连续常量填充、带奇偶谓词的条件矩阵归约，以及保留边框语义的 stencil 内部卷积。匹配过程检查循环初值、步长、界限、唯一前驱、支配关系、元素宽度、stride、对象来源和标量出口；成功后只把入口分支替换为携带原 header、exit、边界和对象操作数的 `aarch64.neon.*` pseudo。若静态结构不满足，Machine IR 原样保留；需要运行期检查的场景由 pseudo 先执行 guard，失败时回到原标量 CFG。

阶段排序所维持的不变量

- **语义不变量：**优化只读取 typed Machine IR、CFG、use/def、对象身份和立即数，块名与用户符号仅作为引用，不参与模式判定。
- **ABI 不变量：**参数寄存器、caller/callee-saved、`x29`/`x30`、scratch 集合和 16 字节栈对齐统一来自 AArch64 target 描述。
- **回退不变量：**结构证明失败时不改写；带动态前提的向量路径显式保留 scalar header、fallback 与 exit，因而不会把性能假设升级为语义假设。
- **打印边界：**AsmPrinter 不再重新发现循环或依赖标签拼写，只确定性展开已经由 Machine IR pass 证明并编码的 pseudo，同时完成立即数和寻址模式的合法化。

表 2.1: AArch64 后端各阶段的输入与主要作用

阶段	关键实现	作用
目标降低	Target lowering	把 IR 值、对象、Phi、调用和 CFG 转为 typed Machine IR，保留虚拟寄存器和帧索引。
地址与循环规范化	AddrReassoc + Machine LSR	提升嵌套循环中的不变地址，把 $base + iv \times stride$ 改写为预头初值、header Phi 和回边定步长指针更新。
SSA 与控制流消解	Phi + compare-branch	在前驱边插入并行 copy，必要时拆分临界边；把唯一使用的 `icmp+br` 融合为 `br.icmp.*`。
局部内存与指令合并	Stack promotion + A64 combine	将未逃逸的整型标量栈对象提升为虚拟寄存器；再次折叠新暴露的地址；形成 MADD/MSUB、compare-select 和 indexed load/store。
分配与物理寄存器清理	Graph-coloring RA + DCE	分整数/浮点类建立冲突图，按 ABI 偏好着色并插入 spill；随后依据跨块活跃性删除死亡的物理寄存器定义。
栈帧与布局	Frame/Stack + layout	计算传出参数区、局部对象和保存区，生成 16 字节对齐的序言/尾声；重排自然循环与 trace，使热后继尽量成为 fall-through。
物理寄存器后优化	Post-RA passes	利用已确定的寄存器、帧偏移和 CFG 形成 LDP、SUBS 分支、尾递归摘要以及受 guard 保护的 NEON pseudo。
汇编降低	AsmPrinter	展开普通机器指令和 `aarch64.*` pseudo，处理立即数、地址、调用约定与段信息，输出 GNU AArch64 汇编。

第 3 章

算法级循环与数组优化

3.1 二维仿射更新的行摘要降维

考虑二维更新

$$D_{i,j} \leftarrow D_{i,j} \cdot F_{i,k} + S_{k,j} \pmod{2^{32}}. \tag{3.1}$$

若后续只观察整行和，则利用模环上的分配律：

$$R_i = \sum_j D_{i,j}, \qquad R_i \leftarrow R_i \cdot F_{i,k} + \sum_j S_{k,j} \pmod{2^{32}}. \tag{3.2}$$

通过仅维护各行元素之和，而不再更新完整的二维矩阵，可将每次调用的时间复杂度从 $O(n^3)$ 降至 $O(n^2)$。该优化是 01_mm 数量级收益的主要来源。

3.2 幂等迭代与最后可观察迭代

对于循环状态更新的过程

$$S_{t+1} = F(S_t),$$

如果能够证明

$$F(F(S)) = F(S),$$

那么 F 具有幂等性。此时，只要循环至少执行一次，多次迭代的结果就与执行一次相同，因此可以将循环次数 N 简化为

$$N > 0 ? 1 : 0.$$

当 $N \le 0$ 时，循环仍然不执行，因此该变换不会改变原程序的边界行为。

目前可识别的情况主要有两类：一类是循环体在每次迭代中都会完全覆盖相关状态；另一类是每次迭代都先从同一个循环不变量复制出完整的初始状态，再调用结果确定且无副作用的函数。在这两种情况下，前面迭代产生的结果都不会影响最终结果。

倒计数循环的 `dead-prefix` 优化处理的是另一种情况：每次迭代都会完全覆盖上一轮产生的状态，而循环结束后只使用最后一次迭代的结果。对于初值大于 0 的倒计数循环，最后一次迭代开始时，归纳变量的值一定为 1。因此，可以将循环初值改写为

$$iv > 0 ? 1 : iv,$$

从而跳过不会影响最终结果的前置迭代。

3.3 重复仿射复制的按需反向解析

当程序连续执行多次仿射数组复制，而后续代码只读取目标数组的一小段前缀时，完整执行所有复制会产生大量无效计算。为此，编译器只从实际会被读取的目标下标出发，按照复制操作的相反顺序追踪数据来源，找到决定该位置最终值的最后一次写入。

在分析同一次复制操作时，还需要考虑写入之间的先后关系。如果某个源位置已经在本次复制的较早阶段被覆盖，那么后续读取到的应当是覆盖后的新值，而不是复制开始前的旧值。通过这种方式，编译器可以直接确定每个目标元素最终对应的源元素，而不必真正执行中间的所有复制过程。

优化后的快速路径会在运行时检查数组长度、容量和行宽等条件。只有在这些条件满足时才采用简化后的计算；如果检查失败，则退回原始复制循环，从而保证优化前后的程序语义一致。

该优化本质上是一种由实际读取需求驱动的“最后写入者切片”。如果被复制的数据最初由一个无副作用的初始化循环生成，还可以继续追踪到初始化表达式。此时，编译器能够根据最终解析出的源下标直接计算对应的初始值，从而同时省去初始化循环和中间复制操作。

3.4 循环规范化与局部变换

系统还实现以下通用循环能力：

- 单调 continue 后缀提前退出与单调上界收紧；
- 内层循环不变量保护链外提；
- 仿射/模递推闭式化、有界余数前缀扫描、商区间块总结和 floor-sum 超块；
- 1–8 次小行程纯递推展开与模仿射表达式规范化；
- 10 路标量归约 unroll-and-jam 与跨 lane 地址共享；
- 跨迭代 Store-to-Load 转发、循环标量提升和 compare-select if-conversion；
- 局部数组死初始化裁剪、提前返回后的初始化下沉、不可观察全局数组死写删除。

第 4 章

有序矩阵计算与 AArch64 微内核

4.1 为什么不能直接套用 GEMM

这些热点循环都采用原地更新的三角递推方式。计算第 i 行时，前面的行已经完成本轮更新，而后面的行仍保留更新前的值，因此同一次计算会同时依赖新、旧两个版本的数据。若简单交换 $i/k/j$ 三层循环的顺序，或让多个输出行长期保存在寄存器中，就可能破坏原有的读写顺序，从而改变程序语义。

我们曾在 Cortex-A53 上测试 `output-stationary` 调度，但性能反而下降了近两倍。这说明，减少 `load/store` 次数并不一定能够提升性能，更不能忽略程序原有的数据依赖和执行顺序。

因此，本系统在处理每个面板时，显式区分以下三类数据来源：

$$\text{source}(k) = \begin{cases} \text{new row } k, & k < i, \\ \text{panel snapshot}, & i \le k < i + B, \\ \text{old row } k, & k \ge i + B. \end{cases} \tag{4.1}$$

每个目标行内部仍按原 k 次序累加。

4.2 多级有序面板

表 4.1: 有序面板设计与寄存器权衡

面板	结果向量	数据复用	适用与回退
2x16	8 个	两行共享前缀/后缀源向量	列数可被 16 整除；奇数尾行回到单行路径
4x8	8 个	四行共享 8 列源面板	以较窄列宽换取更多跨行复用
4x16 lazy	16 个	旧行按依赖点读取，不复制完整矩阵	列数可被 16 整除；寄存器压力较高但官方有效
8x8 lazy	16 个	八行共享源面板，结果峰值不高于 4x16	当前接受的更高复用路径；不足八行回退

统一后缀贡献折叠先把可证明的常量行后缀汇总为标量初值，面板只处理真正变化的前缀。最终 Store 与平方和归约融合，使结果仍在向量寄存器时直接累加，删除后续完整二维读取扫描。

4.3 Wavefront、列打包与预取

LU/三角消元类计算采用 wavefront 调度：先完成当前上三角行，再把已完成列打包到连续栈缓冲，随后更新当前列以下各行。跨行点积由两条连续流组成，可复用 8-lane NEON reduction。K8xJ32 二维分块限制活跃源行为八行，使目标行与源小块保持在 L1。

对每个 32 元素行更新，后端从参数类型得到真实行字节跨度，并发出两条 `prfm pldl1keep` 预取下一行的两个 cache line。该方案不增加地址计算，官方 Cortex-A53 上 h-5 三项合计由 15.76s 降至 11.82s，下降约 25%。

4.4 条件矩阵归约与 parity unswitch

对于低位奇偶谓词控制的矩阵乘加，编译器证明两个输入对象与输出对象分别对应不同的可识别存储对象，因而不存在输入与输出之间的对象级别名；同时证明输出位置只有唯一的归约写入、每个输出元素的 k 顺序保持，然后将连续 j lane 向量化。这里的对象互异是 Machine IR 中地址来源和全局对象身份的别名证明，不是对运行时输入值或样例模式的识别。固定 k 时，一个谓词操作数在整行不变：若为偶数，谓词恒真并走无掩码路径；若为奇数，只需检查另一向量操作数的低位。主循环支持 4/8/16/32/64 列层级及精确尾块。

第 5 章

递归、状态摘要与记忆化

5.1 递归优化分层

表 5.1: 递归与状态优化的分层策略

策略	证明条件	收益机制
尾递归消除	递归结果直接返回，调用后无可观察操作	将递归转换为循环，消除栈增长
受预算递归展开	直接自递归、具有完整的 CFG 和 Phi，静态增长不超过预算	减少函数调用和分支层次
完整参数键记忆化	含 1–3 个整数参数，调用链无外部写副作用	将指数级重复状态转换为有界哈希查表
尾递归增量摘要	两个 i32 参数、纯加法 depth 和单一只读 context	缓存剩余增量，使不同初始 depth 共享结果
只读 helper 阶段缓存	单一循环调用点，所读取的全局状态在阶段内保持不变	跨循环迭代复用纯函数结果
动态固定点记忆化	隐藏状态在函数入口和出口处运行时相等	跳过后续完全相同的状态转移

5.2 尾递归增量摘要

若递归函数满足

$$f(s, d, c) = d + \Delta(s, c), \tag{5.1}$$

则缓存不需要绑定初始值 d，只需保存 $\Delta + 1$，并用零值表示空槽。一般路径以 (s, c) 作为完整键，使用 2^{20} 个状态和 8 字节条目。若能够证明 context 在程序运行期间只初始化一次且不会逃逸，则可将条目压缩为 4 字节，并将状态窗口扩展到 2^{24}。绝对终止值由独立的 1 字节旁表记录，不能与加性摘要混合存储。

该优化要求递归过程不存在额外的 call 或 store，context 的读取位置唯一，`depth` 只参与单位递增和加性返回。同时，缓存窗口之外必须完整回退到原始实现，哈希冲突或稠密槽冲突也不能改变程序结果。Huffman helper 的闭式化、晚期内联和状态本地化相结合后，在官方 Cortex-A53 上将运行时间由约 13.92 s 降至 7.67 s。

5.3 闭式 helper 与状态本地化

位累加 helper 根据真实 CFG 枚举两个布尔输入的四种取值组合（00、01、10、11），建立真值表，从而识别 AND、OR 和 XOR，而不是依赖函数名称进行匹配。对于有符号输入，需要分别处理负数余数、绝对值以及 INT_MIN；连续的二次幂分派可转换为带范围检查的变量移位，动态有符号除法则使用满足向零截断语义的偏置公式。

helper 缩小后再次执行内联，并依次运行 GlobalScalarLocalization、Mem2Reg、CSE 和 DCE，使多个全局状态只在函数入口加载、在函数出口写回。

第 6 章

Machine IR 与 AArch64 代码生成

6.1 图着色寄存器分配

寄存器分配按整数和浮点类独立构建冲突图，普通函数使用压缩位矩阵，大函数切换为稀疏邻接。跨调用活跃值优先分配 callee-saved；调用参数和返回值建立 ABI 偏好；copy、Phi 和调用边搬运建立循环深度加权 affinity。无法着色的值进入统一 spill slot，并在 use/def 点插入 reload/store。分配后依据精确调用点活跃集合保存 caller-saved。

6.2 地址与循环机器优化

AddressReassociation 把二维地址拆成外层不变基址与内层变化偏移；Machine LSR 把 $base + iv \times stride$ 转为预头初始指针、header Phi 和回边固定步长更新。支持步长一/二、非二次幂跨行、仿射 $iv + c$ 和多个相邻访问共享 pointer Phi。代表性官方结果 InstructionCombining 包含 MADD/MSUB、compare-select、counted branch、相邻 i32 load 配对、连续栈零范围、二次幂除余和 post-RA 物理寄存器重命名。每项规则都处理目标/源重合、立即数编码、scratch 冲突、位宽和 AAPCS64。

6.3 结构化 post-RA 向量化

`AArch64PostRAVectorization` 在寄存器分配、栈降低和布局后检查 typed Machine IR：

- 连续常量填充：4/16 元素 NEON 主循环，双目标运行时区间不别名；
- 条件矩阵归约：结构化对象、CFG、stride 和 parity 证明；
- stencil interior：边框保留标量路径，内部四列使用 LDR/DUP/MLA/STR；
- 所有 pseudo 显式携带原 scalar header、fallback、exit 和对象 operand。

第 7 章

完整优化体系

本章给出当前默认 `-O1` 能力的分类总览。详细 pass 索引见附录附录 A。

7.1 过程间、函数与状态优化

普通函数内联、常量实参传播、尾递归消除、直接递归展开、完整键递归记忆化、只读 helper 缓存、状态调用链整链内联、全局标量本地化、过程间效果分析、确定性状态转移固定点和关联哈希链聚合。

7.2 循环与算术优化

常量折叠、CSE、DCE、LICM、循环不变量 guard 外提、仿射模递推、幂等迭代、dead-prefix、单调 continue/上界、Div/Rem 配对、余数比较、模仿射规范化、有界递推展开、有界余数前缀、商区间总结、floor-sum 超块、unroll-and-jam。

7.3 数组、内存与布局优化

数组摘要降维、仿射拷贝传播、lazy affine 解析、初始化值转发、producer-consumer 融合、Store-to-Load 转发、循环标量提升、转置影子表、wavefront 列打包、行更新换序、有序面板、统一后缀、store-to-reduction 融合、死全局数组 Store、局部数组初始化裁剪/下沉、冗余 Load/Store 消除。

7.4 AArch64 目标优化

宽模乘、整数点积向量化、纯映射递推向量化、常量余数向量化、地址重关联、pointer induction、MADD/MSUB 与 shifted register、CSEL、CBZ/CBNZ、LDP/STP、NEON 常量填充、条件矩阵、stencil interior、行更新预取、栈零范围和 post-RA 结构化尾摘要。

第 8 章

实验设计与性能结果

8.1 评测方法

表 8.1: 验证与性能证据层级

层级	用途	判定原则
匿名合成测试	边界、别名、零趟、回绕、fallback 和结构变体	必须与 O0 或关闭版逐字对拍
140 correctness	前端、ABI、后端和共享路径正确性	当前版本 140/140 AC
60 performance	全量编译、汇编、静态链接、执行与输出	当前本地 57 AC + 3 个既有 h-4 格式例外
QEMU / 静态汇编	动态工作量、热点和方向诊断	不能覆盖真实平台退化
官方 Cortex-A53	最终执行时间与正确性	唯一正式性能接受依据

本轮 Machine IR 合规迁移使用提交 `46351fd` 作为独立汇编基线，当前提交为 `0fc1bc5`。完整 200 份 AArch64 汇编均已由 GNU assembler 接受。

8.2 代表性官方结果

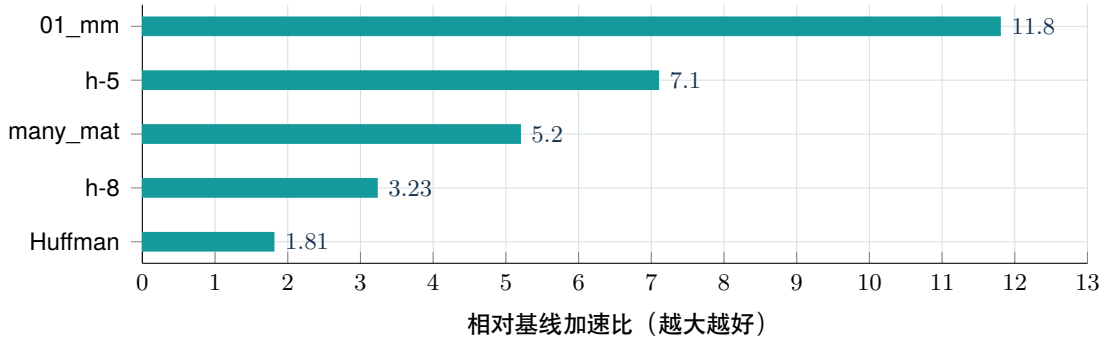


图 8.1: 代表性官方 Cortex-A53 结果。h-5 与 many_mat 为同一接受分支上的累计演进。

表 8.2: 重点优化结果与技术归因

样例族	基线 → 优化后	改善	主要技术
01_mm	约 4.0 s → 0.34 s	约 11.8×	二维仿射更新的行摘要降维与系数后缀总结
h-5	27.99 s → 3.94 s	约 7.1× 累计	8-lane 归约、Machine LSR、K8×J32、wavefront、列打包和 PRFM
many_mat	约 12.4 s → 2.37 s	约 5.2×	统一后缀，以及 2×16、4×8、4×16、8×8 有序 lazy 面板与融合归约
h-8	34.35 s → 10.64 s	3.23×	循环标量驻留、转置影子表和必经路径合法性分析
Huffman	13.92 s → 7.67 s	1.81×	bit helper 闭式化、晚期内联和全局状态本地化
h-4	35.47 s → 27.72 s	21.85%	有界 signed-remainder 的 32 项前缀扫描
matmul	14.06 s → 12.82 s	8.82%	条件矩阵的 16 列主循环与 parity unswitch

8.3 代表性优化案例分析

8.3.1 01_mm：从指令优化到算法降维

该案例不是把某个循环展开得更宽，而是证明最终观察只需要行摘要，因而把完整二维状态递推替换为一维递推。优化仍保留别名、额外读取、部分清零和未知调用 fallback。这说明编译器能够利用跨函数、跨循环的观察等价关系，而不是按样例身份替换算法。

8.3.2 h-5：围绕真实缓存层级逐步演进

演进路径包括四列/八列 NEON、跨行 pointer IV、两路标量交错、counted loop、K8xJ32 分块、wavefront 列打包和下一行预取。期间也出现多行固定寄存器内核、系数 by-element MLS 等官方退化候选，均被独立回退。最终收益来自减少算法访存、形成连续流并隐藏 cache miss，而不是盲目扩大 SIMD 宽度。

8.3.3 many_mat：有序依赖下的面板化

普通 output-stationary 调度在官方平台接近 2 倍退化，原因是破坏连续行访问和原地数据版本。接受方案通过 lazy snapshot 精确选择新/旧行，并保持每个输出元素的归约顺序；面板形状随寄存器预算逐步演进。该案例体现了**依赖正确性、缓存局部性和寄存器压力**三者必须联合设计。

8.4 负结果如何反向约束设计

表 8.3: 真实平台否决的代表候选及所得结论

候选	官方结果	对后续设计的约束
常量 magic division	h-4 约退化 28%	A53 的硬件 SDIV、依赖链和代码尺寸必须进入成本模型
条件矩阵 output-stationary	约退化 99%	保持连续 $k \to j$ 行扫描，不能只看输出 load/store 数
四行 packed 点积	约慢 2.36x	固定向量寄存器数量和大基本块会压垮发射/取指
K8 系数常驻 by-element MLS	约退化 15.7%	删除 scalar load/dup 不代表 by-element SIMD 更快
尾缓存省略一条旁表 Store	约退化 1.34%	微小静态减少不足以覆盖布局和调度扰动

8.5 当前版本验证结论

表 8.4: 提交 `0fc1bc5` 的验证结果

验证项目	结果
构建环境	Docker Release 构建成功，编译器位于 `/workspace/build/compiler`。
AArch64 correctness	140/140 AC。
Performance	57 AC；其余 3 项为父版本已有的 h-4 末尾格式例外，且 h-4 汇编未发生变化。
汇编差异	迁移前后的 60 份汇编中有 57 份逐字一致，仅 h-1 的三份汇编各减少一条分支。
汇编器验证	200 份最终 AArch64 汇编均通过 GNU assembler。
后端合规性	不存在通过汇编文本正则或生成标签拼写触发优化的路径。

第 9 章

总结与后续工作

本项目的主要价值在于：把教学型编译器常见的局部优化，扩展为可证明的跨循环、跨函数状态变换，并将这些变换落到面向 Cortex-A53 的 Machine IR、寄存器、NEON、缓存和预取设计上。优化不依赖样例身份；失败候选由真实平台否决并隔离回退。

后续工作重点不是继续增加专项模板，而是建设共享的 DominatorTree/LoopInfo、MemorySSA、SCEV 风格范围、统一 FastPath/Guard/Fallback 重写器，以及能同时估计代码尺寸、寄存器压力、load/store、NEON 发射和 cache 的 A53 成本模型。这些基础设施将把当前多个成功但独立的结构证明进一步泛化。

第 A 章

默认 -O1 Pass 完整索引

表 A.1: 当前 Driver 中启用的中端 Pass（按功能归类）

Pass	作用
`Mem2RegPass`	局部标量槽提升为 SSA，后期支持动态初值循环状态槽。
`ConstantFoldingPass`	常量算术、比较、分支和 select 折叠。
`FunctionInliningPass`	受预算的小函数完整 CFG 内联。
`ConstantArgumentPropagationPass`	全调用点常量共识传播。
`TailRecursionEliminationPass`	直接尾递归转循环。
`WidenedModularMultiplyPass`	AArch64 安全范围内的宽模乘。
`RecursiveInliningPass`	受代码增长约束的直接递归展开。
`AssociativeChainAggregationPass`	关联哈希重复链聚合。
`RecursiveMemoizationPass`	完整整数参数键递归记忆化。
`GlobalScalarLocalizationPass`	全局标量入口加载、出口回写。
`BitwiseIdiomRecognitionPass`	基于 CFG 真值表的逐位 AND/OR/XOR 闭式。
`PowerOfTwoDispatchPass`	连续幂二分派归一化。
`AffineModularRecurrencePass`	仿射及模递推闭式化。
`ArraySummaryDimensionCollapsePass`	二维仿射更新折叠为行摘要。
`IdempotentIterationCollapsePass`	幂等状态变换的冗余迭代消除。
`DeadPrefixIterationEliminationPass`	倒计数循环只保留最后可观察迭代。
`AffineArrayCopyPropagationPass`	完整二维仿射复制传播。
`CommonSubexpressionEliminationPass`	支配范围公共子表达式消除。
`LoopInvariantCodeMotionPass`	安全循环不变量外提。
`LoopInvariantGuardHoistingPass`	内层循环不变量保护链外提。
`AdjacentProducerConsumerLoopFusionPass`	相邻二维 producer/consumer 融合。
`LoopUnrollAndJamPass`	带依赖证明的标量归约多 lane 展开合并。
`CrossLoopStoreForwardingPass`	完整生产 Store 向后续归约转发。
`DivRemPairStrengthReductionPass`	复用余数恢复二次幂商。

Pass	作用
RemainderComparisonPass	二次幂余数比较与奇偶化简。
RedundantLoadEliminationPass	冗余 Load 消除。
RedundantStoreEliminationPass	冗余 Store 消除。
BoundedModuloReductionPass	支配范围保护下的余数强度削弱。
ModularAffineCanonicalizationPass	i32 模环仿射表达式规范化。
BoundedRecurrenceExpansionPass	小行程纯递推直线展开。
LoopWavefrontColumnPackingPass	波前列打包。
LoopInterchangeRowUpdatePass	有序行更新换序、分块与面板生成。
AArch64IntegerDotReductionPass	AArch64 整数点积 NEON/标量交错。
MonotonicContinueEliminationPass	单调无副作用 continue 后缀删除。
MonotonicLoopBoundPass	单调嵌套上界收紧。
LoopCarriedStoreToLoadForwardingPass	跨迭代相邻 Store-to-Load 转发。
LoopScalarPromotionPass	带仿射不别名证明的循环标量驻留。
TriangularTransposeShadowPass	下降三角 DP 转置影子表。
AArch64PureMapRecurrenceVectorizationPass	有序状态中的纯映射 NEON 并行化。
AArch64VectorConstantRemainderPass	连续 i32 常量余数向量化。
NestedInvariantReductionPass	跨外层重复的只读不变归约折叠。
StoreProducerReductionFusionPass	最终 Store 与平方和归约融合。
StackAggregateInitPruningPass	完整覆盖前的局部数组死初始化裁剪。
GuardedInitializationSinkingPass	提前返回守卫后的初始化下沉。
DeadGlobalArrayStoreEliminationPass	不可观察全局数组对象级死写删除。
DeadCodeEliminationPass	无使用纯指令和不可达结果清理。
TrivialBlockEliminationPass	空块和跳板块清理。
LazyAffineCopyResolutionPass	重复仿射复制按需逆向解析及初始化转发。

第 B 章

后端阶段与关键开关

B.1 后端阶段

AddressReassociation、MachineLoopStrengthReduction、PhiElimination、CompareBranchFusion、StackObjectPromotion、AArch64InstructionCombining、RegisterAllocator、MachineDeadCodeElimination、FrameLowering、StackLowering、MachineLoopBlockPlacement、MachineTraceBlockPlacement、AArch64InstructionCombining::runPostRA、AArch64LateMachineOptimization、AArch64PostRAVectorization 和 AArch64AsmPrinter。

B.2 代表性消融开关

开关	作用
COMPILER_DISABLE_RECURSIVE_MEMOIZATION	关闭递归完整键记忆化
COMPILER_DISABLE_COMPACT_TAIL_MEMOIZATION	关闭紧凑尾摘要 context 证明
COMPILER_DISABLE_IDEMPOTENT_ITERATION_COLLAPSE	关闭幂等迭代折叠
COMPILER_DISABLE_DEAD_PREFIX_ITERATION_ELIMINATION	关闭最后迭代保留
COMPILER_DISABLE_TRIANGULAR_TRANSPOSE_SHADOW	关闭转置影子表
COMPILER_DISABLE_LOOP_SCALAR_PROMOTION	关闭循环标量驻留
COMPILER_DISABLE_LAZY_AFFINE_COPY_RESOLUTION	关闭仿射复制按需解析
COMPILER_DISABLE_LOOP_INVARIANT_GUARD_HOISTING	关闭 guard 外提
COMPILER_DISABLE_AARCH64_STENCIL_INTERIOR_VECTORIZATION	关闭 stencil interior 向量化
COMPILER_AARCH64_DISABLE_CONDITIONAL_MATRIX_REDUCTION	关闭条件矩阵向量化
COMPILER_DISABLE_AARCH64_ROW_UPDATE_PREFETCH	关闭行更新预取

COMPILER_DISABLE_AARCH64_STP_LDP	关闭 post-RA 栈访问配对
COMPILER_DISABLE_AARCH64_SUPEROPT_MUL	关闭 post-RA 常量乘法合成

参考资料与实现披露

[1] S. S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997.

[2] K. Kennedy and J. R. Allen, *Optimizing Compilers for Modern Architectures*, Morgan Kaufmann, 2002.

[3] A. V. Aho et al., *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.

[4] R. Cytron et al., "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph," *ACM TOPLAS*, 1991.

[5] H. S. Warren, *Hacker's Delight*, 2nd ed., Addison-Wesley, 2012.

[6] Arm Ltd., *Procedure Call Standard for the Arm 64-bit Architecture* (*AAPCS64*).

[7] Arm Ltd., *Arm Architecture Reference Manual for A-profile Architecture*.

[8] LLVM Project, *Loop Vectorization and AArch64 Target Transform Information*, public documentation and source behavior.

AI 辅助与独立实现声明

开发过程中使用 OpenAI Codex 辅助源码审查、结构分析、公式推导、C++ 实现、测试设计、汇编差异检查和文档编写。参赛队员负责提出需求、审查修改、运行本地与官方评测并决定保留或回退。本文所述优化基于经典编译理论、公开 AArch64 ISA/AAPCS64 资料和本项目 IR/Machine IR 的独立实现；未复制外部编译器源码，也未把样例名称、公开输入输出、源码指纹或评测环境作为优化触发条件。